
26
EMBARCADERO CONFERENCE
PATROCINADOR MASTER:
embarcadero
An Idera, Inc. Company

Entendendo o Versioning

MVCC / MGA em Bancos de Dados

Multi-Version Concurrency Control
Multi-Generational Architecture



Conceitos, Implementações e Comparações

Carlos Henrique Cantu
firebase.com.br | firebirdnews.org | firebirddevelopersday.com.br

01

Por que Controle de Concorrência?

O problema fundamental dos acessos simultâneos a dados

Exemplo de Problema com Concorrência

Imagine dois usuários acessando a mesma conta bancária ao mesmo tempo:

1. Saldo inicial: R$ 1.000
2. Usuário A lê o saldo: R$ 1.000
3. Usuário B lê o saldo: R$ 1.000
4. Usuário A deposita R$ 500 -> Saldo = R$ 1.500
5. **Usuário B saca R$ 200 -> Saldo = R$ 800 (!)**
6. **Saldo correto deveria ser R$ 1.300, mas ficou R$ 800**



Lost Update

R$ 500 desapareceram!

Sem controle de concorrência:

- Atualizações perdidas
- Decisões tomadas com dados desatualizados
- Saldo final inconsistente
- Sobrescrita de alterações feitas por outra transação

Anomalias de leitura em transações/isolamentos

Dirty Read

Uma transação lê dados que outra transação ainda não confirmou (committed). Se a outra fizer rollback, os dados lidos nunca existiram de verdade.

Non-Repeatable Read

A mesma transação lê **o mesmo registro** duas vezes e encontra valores diferentes, porque outra transação alterou e confirmou esse registro no intervalo.

Phantom Read

A mesma transação **executa novamente uma consulta com o mesmo filtro** e encontra **um conjunto diferente de registros**, porque outra transação inseriu, removeu ou alterou registros e confirmou essas mudanças no intervalo.



A teoria e os exemplos apresentados são baseados na implementação do Versioning no Firebird

Os conceitos podem variar em outros bancos de dados que utilizam MVCC

02

Locking vs Versioning

As duas principais abordagens para controle de concorrência

Abordagem Lock-Based (Pessimista)

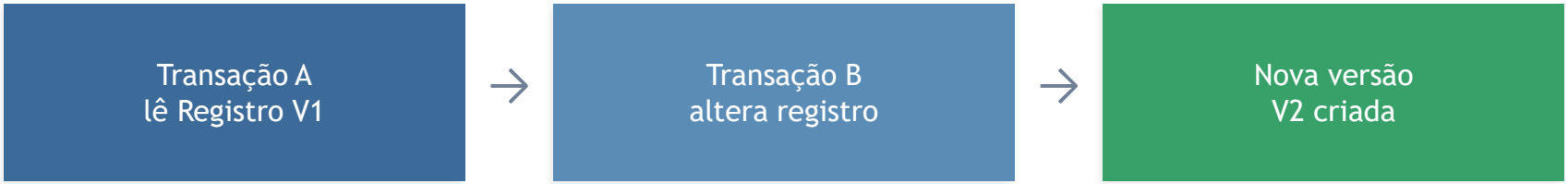
O banco trava (bloqueia) os dados enquanto uma transação está trabalhando com eles.



Problema: Leitores bloqueiam escritores e escritores bloqueiam leitores. Em cenários com muitos acessos simultâneos, o desempenho despenca.

Abordagem Versioning / MVCC

Em vez de bloquear, o banco cria novas versões dos dados. Cada transação enxerga sua própria 'fotografia' consistente.



Transação A continua lendo V1

Sem bloqueio! A transação ve um **snapshot** consistente do momento em que começou.

Transação B trabalha com V2

A nova versão só fica visível para outras transações após o COMMIT.

Princípio fundamental: Leitores nunca bloqueiam escritores. Escritores nunca bloqueiam leitores.

⚖️ Locking vs Versioning: Comparação Direta

Característica	Lock-Based	Versioning (MVCC)
Leitura vs Escrita	Leitores bloqueiam escritores	Nenhum bloqueio mútuo
Overhead	Gerenciamento de locks	Armazenamento de versões
Deadlocks	Frequentes sob carga	Raros ou inexistentes
Leituras longas	Podem causar timeouts	Nunca prejudicadas
Rollback	Desfaz alterações in-place	Descarta versão criada
Lixo/Cleanup	Nenhum necessário	Garbage Collection necessário
Complexidade	Menor (conceito simples)	Maior (gestão de versões)

03

Como Funciona o Versioning

Cadeia de versões, visibilidade e snapshots

O Conceito Central do Versioning

Nunca sobrescrever. Sempre criar uma nova versão.

Quando uma transação modifica um registro, o banco cria uma nova versão em vez de alterar a existente.
A versão antiga permanece disponível para outras transações que ainda precisam dela.

No Firebird (MGA - Multi-Generational Architecture):

- Cada registro possui metadados de controle (Transaction ID de criação, ponteiro para versão anterior)
- As versões antigas formam uma cadeia ligada (back versions) armazenada nas próprias páginas de dados
- A Transaction Inventory Page (TIP) rastreia o estado de cada transação: ativa, committed, rolled back ou limbo
- Cada transação consulta a TIP para decidir qual versão do registro deve enxergar

Cadeia de Versões (Version Chain)

Exemplo: atualização do salário de um funcionário



Como a cadeia funciona:

- O ponteiro do registro aponta sempre para a versão mais recente (V3)
- Cada versão possui um ponteiro para a versão anterior (back pointer)
- No Firebird, todas as versões ficam nas páginas de dados (in-line storage)
- O banco percorre a cadeia até encontrar a versão visível para a transação atual

04

Transações e Níveis de Isolamento

O coração do controle de concorrência

Transações: O Coração do MVCC

No modelo versioning, **tudo gira em torno de transações**. Cada operação acontece dentro de uma transação que define sua visão do banco (isolamento).

A — Atomicidade

Tudo ou nada. Ou todas as operações da transação são aplicadas, ou nenhuma.

C — Consistência

O banco vai de um estado válido para outro estado válido.

I — Isolamento

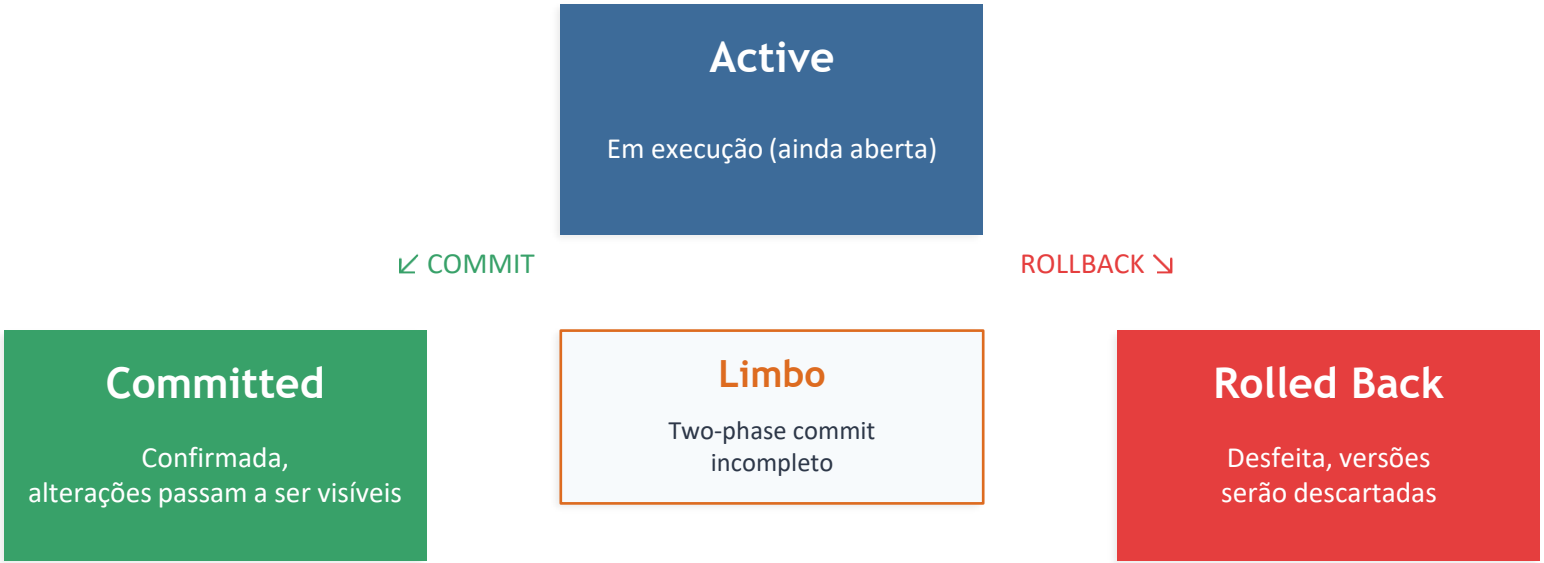
Transações concorrentes não interferem umas nas outras.

D — Durabilidade

Após o COMMIT, os dados sobrevivem a falhas.

No Firebird: Ao nível da API nativa, **toda operação requer transação explícita**. Bibliotecas e drivers de alto nível frequentemente implementam gerenciamento implícito (por conveniência).

Estados de uma Transação no Firebird



Transaction Inventory Page (TIP)

- O Firebird usa apenas 2 bits por transação na TIP para armazenar o estado (active=0, limbo=1, dead=2, committed=3)
- A TIP é consultada para determinar a visibilidade de cada versão de registro
- Marcadores importantes: Oldest Active Transaction (**OAT**), Oldest Interesting Transaction (**OIT**), Oldest Snapshot (**OST**)

Níveis de Isolamento

Nível de Isolamento	Dirty Read	Non-Repeatable	Phantom	Write Skew	Nível equiv. no Firebird
Read Uncommitted	Possível	Possível	Possível	Possível	Não suportado*
Read Committed	Impedido	Possível	Possível	Possível	Sim
Repeatable Read (Snapshot)	Impedido	Impedido	Possível**	Possível***	Sim (SNAPSHOT)
Serializable	Impedido	Impedido	Impedido	Impedido	Sim (SNAPSHOT TABLE STABILITY)

** Firebird nunca permite dirty reads*
*** No Firebird SNAPSHOT, phantoms são efetivamente impedidos pois a visão e congelada no início da transação*
**** Write Skew: duas transações leem os mesmos dados, alteram registros diferentes e comitam, violando a consistência lógica. SNAPSHOT TABLE STABILITY impede esse cenário no Firebird.*

Modos de isolamento no Firebird:

READ COMMITTED: ve dados committed de outras transações a cada statement. Pode usar [NO] RECORD VERSION, ou READ_CONSISTENCY
SNAPSHOT: fotografia do banco no início da transação. Ideal para relatórios.
SNAPSHOT TABLE STABILITY: como SNAPSHOT mas com locks exclusivos nas tabelas acessadas. Impede Write Skew.

Read Committed no Firebird e seus modos

RECORD VERSION

Quando encontra um registro sendo modificado por outra transação, lê a última versão committed sem esperar.

- ✓ Nunca bloqueia leituras
- ✗ Pode ler versão anterior do registro

Ideal para: OLTP de alta concorrência

NO RECORD VERSION

Quando encontra um registro sendo modificado, aguarda a outra transação terminar (commit ou rollback).

- ✓ Sempre lê a versão mais recente
- ✗ Pode causar espera (wait) ou conflitos

Ideal para: operações que exigem dados atualizados

READ CONSISTENCY

Firebird 4+

Snapshot tirado no início de cada statement e mantido até o fim, sem bloqueio.

- ✓ Leitura consistente sem bloqueio
- ✗ Apenas Firebird 4.0 ou superior

Padrão a partir do FB 4, substituindo o RECORD VERSION

Snapshot por statement: No modo Read Committed, o Firebird cria um novo snapshot a cada comando SQL. RECORD VERSION e NO RECORD VERSION podem ver dados diferentes entre SELECTs consecutivos. READ CONSISTENCY (Firebird 4+) mantem o snapshot estável durante todo o statement.

Snapshot (Repeatable Read) no Firebird

Timeline de uma Transação Snapshot



Como funciona

- O Firebird registra quais transações estão ativas no momento do BEGIN
- Só enxerga dados de transações committed antes do BEGIN
- Mudancas de outras transações são invisíveis, mesmo que committed

Quando usar

- Relatorios que precisam de dados consistentes
- Backups lógicos
- Operações longas de leitura
- Cuidado: snapshots longos bloqueiam a coleta de lixo!

Visibilidade das Versões de Registro

Qual versão cada transação enxerga? **Depende do nível de isolamento e do estado da transação que criou a versão**.

Trx #200
(Read Committed)

Ve V2
(última versão comitada, quando Trx #150 comitar, passa a ver V3)

Trx #180
(Snapshot - “foto”)

Ve V2
(committed antes do BEGIN)

Trx #150
(criou V3, ativa)

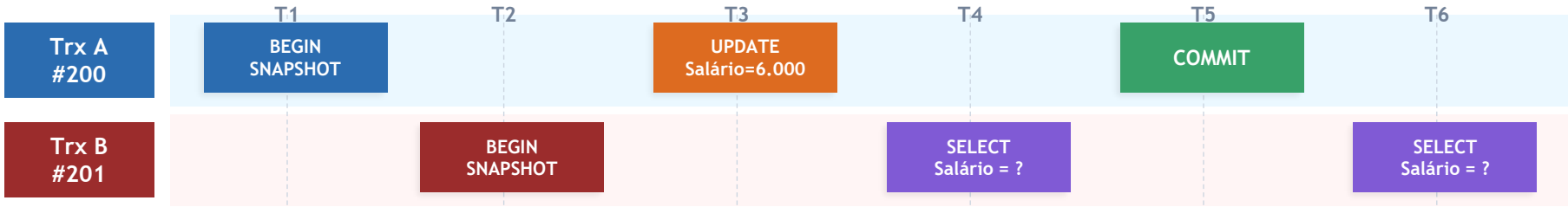
Ve V3
(suas próprias alterações)

Algoritmo de Visibilidade:

1. Le a versão mais recente do registro
2. Consulta a TIP para verificar o estado da transação que criou essa versão
3. Se a transação criadora estiver committed **e** for visível para a transação atual -> usa esta versão
4. Se não, percorre a cadeia de back versions até encontrar uma versão visível

Linha do Tempo: Duas Transações Snapshot

Registro: Funcionário ID=1 | Salário = R$ 5.000 (committed por Trx #100)



Versões do registro em cada momento:

V2 criada	B ve V1	V2 committed	B AINDA ve V1!
Salário: 6.000	Salário: 5.000	Salário: 6.000	Salário: 5.000
(não committed)	(A não committed)	(visível p/ novos)	(snapshot congelado)

Por que B ve R$ 5.000 em T4?

A versão V2 (R$ 6.000) não foi commitada, portanto, nenhuma transação pode enxergá-la.

Por que B AINDA ve R$ 5.000 em T6?

Mesmo após o COMMIT de A, o snapshot de B foi congelado no momento do BEGIN (T2). B nunca vera mudancas posteriores.

E se B tentar fazer UPDATE?

ERRO DE CONFLITO! O Firebird detecta que A já modificou e fez commit desse registro. B precisa fazer rollback e retentar.

First-committer-wins

Update Conflict vs Deadlock no Firebird

Update Conflict

Ocorre quando uma transação tenta alterar/excluir um registro que já foi alterado por outra transação concorrente. Em SNAPSHOT, se a outra transação confirmou uma versão mais nova, a transação antiga não pode gravar sobre ela e recebe erro de conflito.

Exemplo prático:

1. Trx B começa em SNAPSHOT e lê V1
2. Trx A faz UPDATE no registro X
3. Trx A faz COMMIT → cria V2 committed
4. Trx B tenta UPDATE no mesmo registro
5. ERRO: "*deadlock* / update conflict"

Comportamento no Firebird:

- Regra **first-committer-wins**
- Não envolve espera circular entre transações
- Comum em modo **SNAPSHOT**
- Solução: rollback + retry da transação

Deadlock (real)

Ocorre quando duas transações ficam em **espera circular**: A espera B liberar um recurso, enquanto B espera A. Nenhuma pode prosseguir.

Exemplo prático:

1. Trx A faz UPDATE no registro X (segura lock)
2. Trx B faz UPDATE no registro Y (segura lock)
3. Trx A tenta UPDATE no registro Y → espera B
4. **Trx B tenta UPDATE no registro X → espera A = DEADLOCK!**

Comportamento no Firebird:

- MVCC evita locks entre leitores e escritores, mas escritores ainda podem bloquear escritores.
- Possível com WAIT + múltiplos registros em ordem diferente
- Solução: ordenar acesso a registros de forma consistente

Atenção: No Firebird a palavra “deadlock” consta na mensagem de erro para ambos os casos — é preciso analisar o cenário para distingui-los!

⇄ Read Committed vs Snapshot: O Isolamento Muda Tudo

Registro: Funcionário ID=1 | Salário inicial = R$ 5.000 (committed)



O que cada transação enxerga:

	T4 - SELECT (após 1o UPDATE)	T6 - SELECT (após 2o UPDATE)	Por que?
Trx A **READ COMMITTED**	R$ 6.000 ✓	R$ 7.000 ✓	Cada SELECT ve o último valor committed naquele momento
Trx B **SNAPSHOT**	R$ 5.000	R$ 5.000	Snapshot congelado no T2. NUNCA ve mudancas posteriores ao BEGIN

Read Committed = "Sempre atualizado"

A cada SELECT, cria um novo snapshot interno. Qualquer COMMIT de outra transação fica imediatamente visível.

Snapshot = "Fotografia congelada"

Snapshot tirado uma única vez no BEGIN e nunca muda. Ideal para relatórios, mas dados podem estar "defasados".

05

Lixo, Coleta de Lixo e Performance

O lado obscuro do versioning

O Lixo Gerado pelo Versioning

Cada UPDATE ou DELETE cria versões antigas que ficam no banco até serem limpas.

Lixo de Transações Committed

Versões antigas de registros que foram atualizados por transações já confirmadas. Nenhuma transação ativa precisa mais dessas versões.

Limpeza:

Limpo automaticamente quando uma transação subsequente percorre o registro (cooperative GC) ou pelo garbage collector em background.

Lixo de Transações Rolled Back

Versões criadas por transações que foram desfeitas (rollback). Essas versões nunca tiveram validade e precisam ser removidas.

Limpeza:

Também é coletado pelo Cooperative GC e Background GC ao acessar os registros, mas tende a acumular em registros pouco acessados, tornando o sweep mais necessário.

Mecanismos de Coleta de Lixo no Firebird

Cooperative

As próprias transações de usuário limpam as versões antigas ao percorrer os registros durante leituras.

- Simples e previsível
- Não exige thread extra
- Libera espaço no momento em que o lixo é encontrado

- O custo da limpeza cai sobre a transação do usuário
- Consultas podem ficar lentas ao encontrar muito lixo
- Só limpa registros/páginas visitados

Única opção no Classic e SuperClassic Server

Background

A transação que encontra lixo apenas notifica/enfileira o trabalho; uma thread do servidor tenta limpar depois..

- Reduz o impacto imediato sobre a transação do usuário
- Evita que uma consulta pague todo o custo da limpeza
- Ajuda em bases com muito lixo visitado repetidamente

- Depende de páginas/registros serem detectados ou enfileirados
- A limpeza pode atrasar sob carga
- Consome CPU/I/O em segundo plano

Disponível no SuperServer

Combined

Combina limpeza cooperativa com enfileiramento para background GC, tentando equilibrar latência e cobertura.

Melhor dos 2 mundos!

Padrão do SuperServer

O Sweep: Limpeza Profunda

O que é o Sweep?

O sweep percorre TODA a base de dados removendo versões antigas de registros que nenhuma transação ativa precisa mais. É a operação mais pesada de manutenção do Firebird. Só o Sweep avança a OIT.

Quando o Sweep é disparado?

Automático: quando a diferença entre OST e OIT excede o Sweep Interval (padrão: 20.000 transações)

Manual: via gfix -sweep ou executando um backup (gbak faz GC como efeito colateral).

Pode ser desabilitado: Sweep Interval = 0 (requer sweep manual periódico)

Impacto: O sweep pode degradar significativamente a performance enquanto está executando, pois varre todas as páginas de dados. **Em bases grandes, pode durar horas. Estratégia recomendada: desabilitar o sweep automático e agendar sweeps manuais ou backups em horários de baixo uso**.

Impacto do Lixo na Performance

Por que o lixo prejudica:

Cadeias longas

O banco precisa percorrer muitas versões antigas para encontrar a versão correta de cada registro. Quanto maior a cadeia, maior a latência de cada leitura.

Páginas cheias

Versões antigas ocupam espaço nas páginas de dados, causando mais I/O.

Startup mais lento de transações

Muitas transações acumuladas na TIP aumentam o tempo de início de novas transações snapshot e aumentam o consumo de memória.

Sweep pesado

Quanto mais lixo acumulado, mais demorado e impactante será o sweep. Em bases grandes com muito lixo, o sweep pode levar horas e degradar todo o sistema.

Boas práticas:

COMMIT/ROLLBACK rápido | Evitar snapshots longos | Monitorar gap OIT-OAT | GCPolicy=combined

06

Comparação entre Bancos de Dados

Firebird | PostgreSQL | Oracle | SQL Server | MySQL

Onde as Versões São Armazenadas?

Firebird	PostgreSQL	Oracle	SQL Server	MySQL/InnoDB
In-line (páginas de dados)	**In-line (heap tuples)**	**Undo Tablespace (separado)**	**tempdb ou PVS* (version store)**	**Undo Log (rollback segment)**
Versões antigas ficam nas próprias páginas de dados, formando cadeias de back versions.	Cada UPDATE cria uma nova tupla no heap da tabela. A versão anterior permanece como uma tupla antiga até ser removida ou reutilizada pelo VACUUM.	Versões antigas vão para segmentos de UNDO separados. A tabela principal só tem a versão atual.	Versões vão para o tempdb (classico) ou para o PVS no próprio banco (com ADR, a partir do SQL Server 2019). Não é o comportamento padrão.	Similar ao Oracle. Versões antigas ficam nos undo logs do InnoDB, separadas dos dados.
✓ Acesso rápido a versões recentes	✓ Simples e robusto	✓ Tabela sempre limpa	✓ Não polui tabela principal	✓ Tabela principal enxuta
✗ Paginas incham com muitas versões	✗ Table bloat sem VACUUM	✗ Undo pode ficar cheio (ORA-01555)	✗ tempdb pode ser gargalo (mitigado com ADR/PVS)	✗ Purge thread pode atrasar

In-line (Firebird/PostgreSQL): versões ficam junto aos dados -> precisa de garbage collection ativo. **Separado (Oracle/SQL Server/MySQL):** versões ficam em area separada -> menos inchaço nas tabelas, mas precisa gerenciar o espaco de undo.

** SQL Server 2019+: com Accelerated Database Recovery (ADR) ativado, versões ficam no Persistent Version Store (PVS) dentro do próprio banco, eliminando a dependência do tempdb.*

Tabela Comparativa Completa

	Firebird	PostgreSQL	Oracle	SQL Server	MySQL
MVCC nativo	Sim	Sim	Sim	Opcional	Sim (InnoDB)
Versões em	In-line	In-line (Heap tuples)	Undo tblsp	tempdb/PVS	Undo log
GC principal	Coop/BG/Sweep	VACUUM	Undo Mgmt	Auto (Version Cleanup)	Purge
Nome "padrão"	MGA	MVCC	MVCC/SCN	RCSI / Snapshot	MVCC
Dirty Read	Nunca	Nunca	Nunca	Possível, com NOLOCK	Possível (Read Uncommitted)
ID system	ID de 32/48 bits TIP 2 bits (status)	XID (32 bit) xmin / xmax	SCN	XSN / version chain	TRX_ID
Risco	Gap OIT-OAT	Bloat/XID Wrap	ORA-01555	tempdb/PVS	History list

Conclusão: Firebird e PostgreSQL compartilham a abordagem in-line (versões nos dados), enquanto Oracle, SQL Server e MySQL usam areas separadas. No SQL Server 2019+, o ADR permite usar o PVS no próprio banco em vez do tempdb. Cada abordagem tem seus trade-offs entre simplicidade, performance e necessidade de manutenção.

Resumo e Conclusões

- Controle de concorrência é essencial para garantir integridade em ambientes multiusuário
- Versioning (MVCC) elimina bloqueios entre leitores e escritores criando versões dos dados
- O nível de isolamento define quais versões cada transação pode enxergar
- Versões antigas geram lixo que precisa ser coletado para manter a performance
- No Firebird: sweep + GC policy. No PostgreSQL: VACUUM. Cada banco tem sua estratégia
- **Transações curtas e commit rápido são as melhores práticas em qualquer banco com MVCC**
- In-line vs separado: cada abordagem tem trade-offs entre simplicidade e manutenção

Versioning permite bancos modernos atenderem milhares de usuários sem sacrificar a consistência

EMBARCADERO CONFERENCE 26

PATROCINADOR MASTER: embarcadero An Idera, Inc. Company



QUER ME VER NA
#ECON27?
ACESSE O QRCODE E
AVALIE MINHA PALESTRA!

CARLOS HENRIQUE CANTU

@warmbooter

linkedin.com/in/carloshcantu

www.firebase.com.br